

CombatHub

A Full-Stack Combat Sports Ecosystem — Capstone Project Proposal

1. Project Overview

CombatHub is a unified digital platform for the combat sports community, bringing together athletes, coaches, gym owners, and equipment vendors in a single ecosystem. It combines social networking, e-commerce, direct messaging, content sharing, and premium subscriptions — all behind role-based access control.

Purpose: The combat sports world is fragmented. Gear is sold on one site, coaching is booked on another, and community interaction happens across disconnected social media platforms. CombatHub solves this by providing a single platform where each role has tailored tools: athletes shop and connect, coaches manage clients, vendors list products, and gym owners run their business.

2. Technology Stack

Layer	Technology	Purpose
Backend	Django 5.2 + Django REST Framework	REST API, ORM, authentication, permissions
Frontend	React 19 + Vite + Tailwind CSS v4	SPA with animations (Framer Motion, GSAP)
Database	PostgreSQL 16	Primary data store with proper indexes
Cache	Redis 7	Session store, API cache, WebSocket channel layer
Real-time	Django Channels + WebSockets	Direct messaging notifications
Auth	SimpleJWT + Google OAuth	Token-based authentication with social login

3. Architecture

The application follows a decoupled architecture:

- **Backend:** Django REST Framework serves a RESTful API with JWT authentication. Separate Django apps handle accounts, products, chat, gyms, events, reviews, payments, and subscriptions.
- **Frontend:** React SPA communicates with the backend via REST API calls and WebSockets. Protected routes ensure role-based access.
- **Real-time:** Django Channels consumers handle WebSocket connections for notification delivery.
- **Caching:** Redis caches categories, brands, and session data to reduce database load.

4. User Roles & Permissions

Role	Permissions	Limitations
Athlete	Browse shop, add to cart, checkout, create posts, follow users, message, join groups	Cannot sell products or manage gyms
Coach	Post content, manage athletes, access coach dashboard	Cannot buy products, must have premium for dashboard
Gym Owner	Manage gym, post events, manage coaches	Cannot buy products, must have premium
Vendor	List products, manage inventory, view seller orders	Cannot buy products, must have premium
Admin	Full access to Django admin, moderate content, manage users	Cannot use frontend features

5. Key Features

5.1 Social Features

- **Posts & Gallery:** Image/text posts with like and comment systems
- **Community:** User discovery with role filters, search, and follower system
- **Groups:** Private/public groups with invite, join requests, and messaging

- **Bookmarks:** Save posts and gallery items for later

5.2 Marketplace

- **Products:** Multi-image products with variants (size, color), categories, brands
- **Cart:** Full cart system with quantity controls — athletes only
- **Checkout:** M-Pesa and card payment ready, shipping address collection
- **Orders:** Order history with status tracking (pending, paid, shipped, delivered)

5.3 Messaging

- Real-time direct messaging via WebSockets
- Typing indicators and read receipts
- View-once image messages with privacy controls
- Message editing and deletion

5.4 Premium Subscriptions

- 30-day free trial for new sellers
- Automatic 7-day grace period when trial expires
- Self-removal of premium features when grace ends
- Insights visibility toggle for premium users

5.5 Security & Moderation

- **Authentication:** JWT access/refresh tokens, Google OAuth, TOTP 2FA
- **Authorization:** DRF permission classes, custom IsSeller/IsPremium, object-level checks
- **Rate Limiting:** 1000/day authenticated, 30/hr anonymous, 5/hr newsletter
- **Content Moderation:** Regex-based flagging for profanity, violence, hate speech, spam
- **IP Tracking:** SHA-256 hashed IP logs on every request
- **Email Verification:** 6-digit code verification for email changes
- **Phone Verification:** Africa's Talking API for SMS verification

6. Database Design

Key models and relationships:

- **User (Custom):** Extends AbstractUser with role field, display_name, email_verified, totp_enabled
- **Profile:** One-to-one with User — bio, avatar, phone, height, weight, stance, premium fields
- **Post / GalleryItem:** Content with like and comment models (generic relations where applicable)
- **Product / ProductVariant:** Vendor-owned products with multiple variants
- **Cart / CartItem:** User-scoped shopping cart
- **Order:** Checkout result with item snapshots and status tracking
- **Message:** Chat messages with view-once support
- **Group / GroupMember:** Community groups with membership and roles
- **Follow:** User-to-user follower relationship
- **Notification:** Polymorphic notifications (follow, like, comment, premium, etc.)

7. API Endpoints

Category	Endpoints
Auth	register, login, google, totp, password reset, email/phone verify
Users	me, users list, public profile, follow/unfollow
Social	posts CRUD, gallery CRUD, likes, comments, bookmarks
Community	user list, coaches, vendors, search, groups
Shop	products, categories, brands, cart, checkout, orders
Messaging	conversations, messages, edit, delete, view-once
Premium	start trial, check status, setup payment, cancel
Admin	user management, role updates, messaging blocks
Dashboard	coach stats, vendor stats, gym owner stats

8. Development Timeline

Phase	Tasks	Duration
Planning	Database schema design, API planning, wireframes	1 week
Backend Core	User model, auth, profiles, posts, gallery	2 weeks
Frontend Core	Layout, routing, auth flow, home page, community	2 weeks
Marketplace	Products, cart, checkout, orders, vendor dashboard	2 weeks
Messaging	WebSocket setup, real-time chat, notifications	1 week
Premium	Subscription flow, payment setup, gating	1 week
Polish	Animations, responsive design, error handling, testing	1 week

9. Future Roadmap

- Live payment processing (Stripe API, M-Pesa Daraja API)
- Celery workers for async email and notifications
- Mobile app (React Native)
- Event booking and ticketing
- Live training session streaming
- AI-powered content recommendations